

Concepts, Unix/Linux Installation Process, Hardware Requirements for Unix/Linux, Advantages of Unix/Linux, Reasons for Popularity and Success of Linux/Unix Operating System, Features of Linux/Unix Operating System, Kernel, Kernel Functions**,***

Unix/Linux: Concepts, Installation, Hardware Requirements, Advantages, Popularity, Features, Kernel and Kernel Functions

1. Concepts of Unix/Linux

UNIX and **Linux** are powerful operating systems widely used in servers, networking, cloud computing, supercomputers, embedded systems, and software development. UNIX was developed at Bell Labs in the 1970s. Linux was created by **Linus Torvalds** in 1991 as a Unix-like operating-system kernel. Linux distributions such as Ubuntu, Debian, Fedora, Red Hat Enterprise Linux and others combine the Linux kernel with system utilities, libraries and applications.

The basic concept of Unix/Linux is to provide a **multiuser, multitasking, secure and portable operating environment**. Multiple users can work on the same computer, while several programs can execute at the same time. Unix/Linux follows the philosophy that many system resources can be represented and accessed through files. It also provides a powerful command-line interface called the **shell**, through which users can execute commands and automate tasks using shell scripts.

2. Unix/Linux Installation Process

Installing Unix/Linux generally involves the following steps:

1. **Check hardware compatibility:** Ensure that the computer satisfies the distribution's processor, RAM, storage and firmware requirements.
2. **Download the Linux ISO:** Obtain an ISO image of the selected distribution from its official source.
3. **Create bootable media:** Write the ISO to a USB flash drive using an appropriate bootable-media utility.
4. **Boot from USB:** Enter the computer's boot menu or firmware settings and select the USB device.
5. **Start the installer:** Choose options such as language, keyboard layout and network configuration.
6. **Partition the disk:** Select automatic partitioning or manually create partitions. Important data should be backed up before modifying partitions.
7. **Create user account:** Set the computer name, username and password.
8. **Install the operating system:** The installer copies system files and configures the bootloader.
9. **Restart the computer:** Remove the installation USB when requested and reboot.
10. **Update the system:** After installation, update packages and install required drivers and applications.

3. Hardware Requirements for Unix/Linux

Hardware requirements depend on the particular Linux distribution and desktop environment. Generally, a modern Linux system requires:

- **Processor:** 64-bit Intel or AMD processor is commonly recommended.
- **RAM:** About 2–4 GB may be sufficient for lightweight distributions; modern graphical desktops generally benefit from more RAM.
- **Storage:** At least 20–30 GB is practical for a basic desktop installation, although requirements vary.
- **USB/DVD:** Required for installation media when installing from external media.
- **Graphics:** A compatible graphics adapter is required for graphical desktop environments.
- **Network:** Ethernet or Wi-Fi can be used for Internet connectivity and updates.

Lightweight Linux distributions can operate effectively on older computers with relatively limited resources.

4. Advantages of Unix/Linux

Unix/Linux provides many advantages:

- **Open-source:** Linux source code is publicly available under open-source licenses.
 - **Security:** Unix/Linux provides permissions, ownership and authentication mechanisms.
 - **Multitasking:** Many programs can run simultaneously.
 - **Multiuser:** Multiple users can access system resources.
 - **Stability:** Linux systems can operate for long periods with minimal interruption.
 - **Customization:** Users can select different desktop environments, shells and software.
 - **Networking:** Linux has strong built-in networking capabilities.
 - **Automation:** Shell scripts and command-line tools make administration easier.
 - **Portability:** Linux supports many processor architectures and devices.
 - **Cost effectiveness:** Many Linux distributions can be used without purchasing proprietary operating-system licenses.
-

5. Reasons for Popularity and Success

Linux/Unix became popular because of its **reliability, flexibility, security and powerful command-line environment**. Linux is available for servers, personal computers, smartphones, embedded devices and cloud infrastructure. Developers can examine and modify source code, which encourages innovation and customization.

Linux also has a large global community that contributes software, documentation, security fixes and technical support. Organizations can choose distributions and support models

according to their requirements. Linux is particularly important in servers, containers, cloud platforms and high-performance computing.

6. Features of Linux/Unix Operating System

Important features include:

1. **Multiuser:** Multiple users can work on one system.
 2. **Multitasking:** Several processes can execute concurrently.
 3. **Security:** File permissions and user/group management protect resources.
 4. **Portability:** The operating system can run on different hardware architectures.
 5. **Hierarchical file system:** Files are organized in directories and subdirectories.
 6. **Shell:** Users can interact with the system through command interpreters.
 7. **Networking:** Supports common networking protocols and services.
 8. **Process management:** The system can create, schedule and terminate processes.
 9. **Device management:** Hardware devices are managed through kernel and drivers.
 10. **Piping and redirection:** Output from one command can be passed to another command or redirected to files.
-

7. Kernel

The **kernel** is the central and most important component of an operating system. In Linux, the **Linux kernel** acts as an intermediary between application software and computer hardware.

Applications normally do not directly control the CPU, memory or hardware devices. Instead, they request services from the kernel through **system calls**. The kernel then manages these resources and provides controlled access.

A simplified structure is:

User → Applications → Shell/System Libraries → System Calls → Kernel → Hardware

The kernel operates with high privileges and remains active while the operating system is running.

8. Kernel Functions

The major functions of the Unix/Linux kernel are:

1. Process Management

The kernel creates, schedules and terminates processes. It decides which process receives CPU time.

2. Memory Management

It allocates and releases RAM, manages virtual memory and protects one process's memory from another.

3. File-System Management

The kernel provides mechanisms for creating, reading, writing and deleting files and directories and supports different file systems.

4. Device Management

Device drivers allow the kernel to communicate with hardware such as disks, keyboards, network adapters and USB devices.

5. Security and Access Control

The kernel enforces permissions and other security mechanisms to control access to system resources.

6. Networking

It provides core networking functionality, allowing applications to communicate over networks.

7. System Call Management

The kernel provides **system calls** through which programs request operating-system services such as file access, process creation and memory allocation.

8. Inter-Process Communication

The kernel provides mechanisms that allow processes to exchange data and synchronize their activities.

Conclusion

Unix/Linux is a powerful operating-system family based on principles of **multiuser operation, multitasking, security, portability and efficient resource management**. Its open-source ecosystem, strong networking capabilities, stability, flexibility and extensive community support have made Linux especially important in modern servers, cloud computing, development, embedded systems and supercomputing. At the center of Linux is the **kernel**, which manages the CPU, memory, files, devices, networking and system calls and provides the essential connection between software and hardware.

Assuming you mean **Ubuntu 22.04 LTS** (not 22.024), the following covers all the listed **System Security** topics with commands and practical examples.

System Security in Ubuntu 22.04 LTS

System security is the process of protecting a Linux computer from unauthorized physical access, unauthorized users, malicious programs, and improper access to files and services.

1. Physical Security

Physical security protects the computer and its hardware from unauthorized people.

Important measures include:

- Keep the server in a locked room.
- Restrict access to USB devices and boot media.
- Use BIOS/UEFI passwords where appropriate.
- Use a strong Ubuntu login password.
- Enable screen locking.
- Protect backup disks and storage devices.
- Use full-disk encryption such as **LUKS** when installing Ubuntu.

Check disk and partitions

lsblk

Example:

```
NAME      SIZE TYPE
sda       500G disk
└─sda1    512M part
└─sda2   499.5G part
```

For an existing system, avoid changing partitions unless you have a backup.

2. Controlling System Access

Ubuntu controls system access mainly through **users, passwords, groups, SSH, and permissions**.

Create a user

```
sudo adduser student
```

Set/change password

```
sudo passwd student
```

Check current user

```
whoami
```

Display logged-in users

who

Display user information

id student

Example:

```
uid=1001(student) gid=1001(student) groups=1001(student)
```

Lock an account

```
sudo passwd -l student
```

Unlock an account

```
sudo passwd -u student
```

Account locking is useful when an account should temporarily be prevented from authenticating.

3. Restricted Shells

A restricted shell limits what a user can do from the command line. Bash provides **rbash**.

Start restricted Bash

```
rbash
```

or:

```
bash -r
```

A restricted shell can prevent operations such as changing directories or modifying certain shell environment settings.

Example

Create a restricted-shell user configuration:

```
sudo usermod -s /bin/rbash student
```

Check:

```
getent passwd student
```

Important: `rbash` is not a complete security boundary. A determined user may escape a poorly designed restricted environment. For stronger isolation, use containers, VMs, application-specific shells, or other access-control mechanisms.

4. Controlling File Access

Linux uses permissions to control who can read, write, or execute files.

The three basic permission categories are:

Category	Meaning
u	Owner/user
g	Group
o	Others

Permissions:

Permission	Meaning
r	Read
w	Write
x	Execute

Check permissions:

```
ls -l file.txt
```

Example:

```
-rw-r----- 1 student staff 1200 Oct 7 file.txt
```

Here:

- Owner: `rw-`
- Group: `r--`
- Others: `---`

5. File Access Commands

chmod

Changes file permissions.

```
chmod 600 file.txt
```

This gives:

Owner = read + write

Group = no access

Others = no access

Symbolic example:

```
chmod u+x script.sh
```

Add execute permission for the owner.

Remove write permission from others:

```
chmod o-w file.txt
```

Numeric permissions

```
chmod 755 script.sh
```

Meaning:

Owner = rwx

Group = r-x

Others = r-x

chown

Changes file ownership.

```
sudo chown student file.txt
```

Change owner and group:

```
sudo chown student:staff file.txt
```

chgrp

Changes group ownership:

```
sudo chgrp staff file.txt
```

6. Access Control Lists (ACLs)

An **Access Control List (ACL)** provides more detailed permissions than the traditional owner/group/other permission model.

ACLs are useful when you want to give a particular user or group access to a file without changing the normal owner/group arrangement.

Install ACL utilities if necessary:

```
sudo apt update
sudo apt install acl
```

Check ACL:

```
getfacl file.txt
```

Example:

```
# file: file.txt
# owner: student
# group: staff
user::rw-
group::r--
other::---
```

7. Setting ACL Entries

Suppose teacher needs read access to file.txt.

```
setfacl -m u:teacher:r file.txt
```

Check:

```
getfacl file.txt
```

You may see:

```
user::rw-
user:teacher:r--
group::r--
mask::r--
other::---
```

Give teacher read/write access:

```
setfacl -m u:teacher:rw file.txt
```

Give a group access:

```
setfacl -m g:teachers:rw file.txt
```

8. Modifying ACL Entries on a File

The `-m` option modifies an existing ACL or adds a new entry.

For example, change teacher from read-only to read/write:

```
setfacl -m u:teacher:rw file.txt
```

Verify:

```
getfacl file.txt
```

ACL directory example

Give teacher read/write/execute permission on a directory:

```
setfacl -m u:teacher:rwx project/
```

For directories, `x` allows the user to access/traverse the directory.

9. Deleting ACL Entries on a File

Remove the ACL entry for a particular user:

```
setfacl -x u:teacher file.txt
```

Remove a group ACL:

```
setfacl -x g:teachers file.txt
```

Remove all extended ACL entries:

```
setfacl -b file.txt
```

Then verify:

```
getfacl file.txt
```

Note: `setfacl -b` removes extended ACL entries, so use it carefully.

10. Restricting FTP

Traditional FTP is insecure because normal FTP transmits credentials and data without encryption.

For modern Ubuntu systems, prefer **SFTP through SSH** or **FTPS** rather than plain FTP.

Check SSH service

```
sudo systemctl status ssh
```

SFTP is available through OpenSSH:

```
sftp student@server-ip
```

Example:

```
sftp student@192.168.1.10
```

If an FTP service is not required, disable its service rather than leaving it exposed.

For example, if a legacy FTP daemon is installed:

```
sudo systemctl disable --now <ftp-service>
```

Replace <ftp-service> with the actual service name.

Firewall

Check UFW:

```
sudo ufw status
```

If FTP is not required, don't open FTP ports.

11. Securing Superuser Access

The Linux **root** account has almost unlimited control over the system. Therefore, root access must be carefully protected.

Instead of logging in directly as root, Ubuntu normally uses sudo.

Check whether a user has sudo privileges:

```
groups student
```

or:

```
sudo -l -U student
```

Run an administrative command:

```
sudo apt update
```

Ubuntu records sudo activity in system logs.

12. Restricting Root Access

For SSH, direct root login should generally be disabled.

Edit SSH configuration:

```
sudo nano /etc/ssh/sshd_config
```

Set:

```
PermitRootLogin no
```

Then test the configuration:

```
sudo sshd -t
```

If there is no output, the syntax is normally valid.

Restart/reload SSH:

```
sudo systemctl reload ssh
```

Important: Before changing SSH configuration remotely, keep an existing administrative session open so you don't accidentally lock yourself out.

13. Monitoring Superuser Access

Ubuntu provides several ways to monitor administrative activity.

View sudo-related authentication logs

On Ubuntu systems using `/var/log/auth.log`:

```
sudo grep sudo /var/log/auth.log
```

Example:

```
student : TTY=pts/0 ; PWD=/home/student ; USER=root ; COMMAND=/usr/bin/apt update
```

View recent logins

```
last
```

View failed login attempts

```
sudo lastb
```

Depending on the system's logging configuration, these commands may require appropriate log files or privileges.

Check sudo privileges

```
sudo -l
```

This displays the commands the current user is permitted to run through sudo.

14. TCP Wrappers

TCP Wrappers historically provided host-based access control through:

```
/etc/hosts.allow  
/etc/hosts.deny
```

They were commonly used with services linked against **libwrap**, particularly on older UNIX/Linux systems.

However, **modern Ubuntu versions generally do not use TCP Wrappers for most services**, and OpenSSH does not rely on the old `/etc/hosts.allow` and `/etc/hosts.deny` mechanism.

Modern Ubuntu should instead use **UFW/nftables**, SSH configuration, and service-specific access controls.

Check firewall

```
sudo ufw status verbose
```

Enable UFW:

```
sudo ufw enable
```

Allow SSH:

```
sudo ufw allow ssh
```

Allow SSH only from a trusted network, for example:

```
sudo ufw allow from 192.168.1.0/24 to any port 22 proto tcp
```

Deny a specific host:

```
sudo ufw deny from 192.168.1.50
```

Check rules:

```
sudo ufw status numbered
```

Practical System Security Example

Suppose we have a user called student and a protected project file.

Step 1: Create user

```
sudo adduser student
```

Step 2: Create file

```
sudo touch /home/student/project.txt
```

Step 3: Set ownership

```
sudo chown student:student /home/student/project.txt
```

Step 4: Restrict normal permissions

```
sudo chmod 600 /home/student/project.txt
```

Step 5: Give another user read access using ACL

```
sudo setfacl -m u:teacher:r /home/student/project.txt
```

Step 6: Verify ACL

```
getfacl /home/student/project.txt
```

Step 7: Remove teacher's ACL

```
sudo setfacl -x u:teacher /home/student/project.txt
```

Step 8: Check firewall

```
sudo ufw status
```

Step 9: Monitor administrative activity

```
sudo grep sudo /var/log/auth.log
```

Quick Reference

Topic	Ubuntu Command/Method
Current user	whoami
User information	id username

Topic	Ubuntu Command/Method
Logged-in users	who
Create user	sudo adduser username
Change password	sudo passwd username
Lock account	sudo passwd -l username
File permissions	ls -l file
Change permissions	chmod 600 file
Change owner	chown user file
Change group	chgrp group file
View ACL	getfacl file
Set ACL	setfacl -m u:user:r file
Modify ACL	setfacl -m u:user:rw file
Delete ACL entry	setfacl -x u:user file
Remove extended ACLs	setfacl -b file
Check sudo permissions	sudo -l
SSH configuration	/etc/ssh/sshd_config
Disable SSH root login	PermitRootLogin no
Check firewall	sudo ufw status
Allow SSH	sudo ufw allow ssh
Login history	last
Failed logins	sudo lastb
Sudo log	sudo grep sudo /var/log/auth.log

Conclusion

Ubuntu system security combines **physical protection, user/account management, file permissions, ACLs, restricted services, sudo control, SSH hardening, logging, and firewall protection**. For modern Ubuntu 22.04 systems, **ACLs, sudo, OpenSSH, UFW/nftables, and proper logging** are more relevant than older mechanisms such as traditional TCP Wrappers and CDE-style administration.

explain and describe all topic with command and example using Ubuntu 22.024 linux os system “Dynamic Host Configuration Protocol: - Introduction, DHCP Leased Time, DHCP Scopes, DHCP IP Address, Allocation Types, Planning DHCP Deployment, DHCP Configuration files, Automatic Startup of DHCP Server, Configuration of DHCP Clients, Manually Configuring the DHCP.” ?

Assuming you mean **Ubuntu 22.04 LTS** (rather than “22.024”), here is a complete explanation of **Dynamic Host Configuration Protocol (DHCP)** with commands and practical examples.

Dynamic Host Configuration Protocol (DHCP)

DHCP (Dynamic Host Configuration Protocol) is a network protocol that automatically provides computers and other devices with network configuration information such as:

- IP address
- Subnet mask
- Default gateway
- DNS server
- Lease time
- Domain information

On Ubuntu Server, the **Kea DHCP server** is the modern ISC DHCP replacement, while older courses and systems commonly use the isc-dhcp-server package. The examples below use isc-dhcp-server because it directly matches the traditional DHCP syllabus terminology.

1. Introduction to DHCP

Without DHCP, every computer would need to be manually configured.

For example:

IP Address: 192.168.1.100
Subnet Mask: 255.255.255.0
Gateway: 192.168.1.1
DNS: 8.8.8.8

With DHCP, the client asks the DHCP server for configuration.

The basic DHCP process is called **DORA**:

Client → DHCP Discover
Server → DHCP Offer
Client → DHCP Request
Server → DHCP Acknowledgement

Install DHCP server

```
sudo apt update  
sudo apt install isc-dhcp-server
```

Check the service:

```
sudo systemctl status isc-dhcp-server
```

2. DHCP Lease Time

A **DHCP lease** is the amount of time for which a DHCP server gives an IP address to a client.

For example:

```
default-lease-time 600;  
max-lease-time 7200;
```

Here:

- 600 seconds = 10 minutes
- 7200 seconds = 2 hours

Configuration example

Edit:

```
sudo nano /etc/dhcp/dhcpd.conf
```

Add:

```
default-lease-time 600;  
max-lease-time 7200;
```

The client normally attempts to renew its lease before it expires.

3. DHCP Scopes

A **DHCP scope** defines the range of IP addresses that the DHCP server can distribute to clients.

For example:

```
Network:    192.168.1.0/24  
Gateway:    192.168.1.1  
DHCP range: 192.168.1.100 - 192.168.1.200
```

Configuration:

```
subnet 192.168.1.0 netmask 255.255.255.0 {  
    range 192.168.1.100 192.168.1.200;  
    option routers 192.168.1.1;  
    option subnet-mask 255.255.255.0;  
    option domain-name-servers 8.8.8.8 1.1.1.1;  
}
```

The server will allocate addresses between .100 and .200.

4. DHCP IP Address

A DHCP server dynamically assigns an IP address to a client.

For example:

```
Laptop
↓
DHCP Discover
↓
DHCP Server
↓
192.168.1.105
```

On Ubuntu, check the assigned IP address with:

```
ip addr
```

or:

```
ip -4 addr
```

Example:

```
inet 192.168.1.105/24
```

Check the routing information:

```
ip route
```

Example:

```
default via 192.168.1.1
```

5. DHCP IP Address Allocation Types

DHCP generally uses three important allocation methods.

A. Dynamic Allocation

The server temporarily assigns an address from its DHCP pool.

Example:

```
192.168.1.100
192.168.1.101
192.168.1.102
...
```

The address is associated with a lease.

This is commonly used for:

- laptops
 - smartphones
 - desktop computers
 - temporary devices
-

B. Automatic Allocation

The DHCP server assigns an address to a client and may keep that address associated with the client.

This can provide more consistent addressing than ordinary dynamic allocation.

C. Manual/Static DHCP Reservation

The administrator associates a client's MAC address with a specific IP address.

Example:

```
host printer {  
    hardware ethernet 00:11:22:33:44:55;  
    fixed-address 192.168.1.50;  
}
```

The printer will receive:

192.168.1.50

This is useful for:

- printers
 - servers
 - network devices
 - surveillance systems
-

6. Planning DHCP Deployment

Before installing DHCP, plan the network.

Example:

Parameter	Value
Network	192.168.1.0/24
Gateway	192.168.1.1

Parameter	Value
DHCP Server	192.168.1.10
DHCP Range	192.168.1.100–200
DNS	192.168.1.1
Reserved addresses	.1–.99
DHCP clients	.100–.200

Important planning points

1. Identify the network/subnet.
2. Decide the DHCP server's own IP address.
3. Define the DHCP address pool.
4. Reserve addresses for servers and network equipment.
5. Configure the default gateway.
6. Configure DNS servers.
7. Decide appropriate lease times.
8. Avoid having multiple unauthorized DHCP servers on the same network.
9. Plan separate scopes for different networks/VLANs.

7. DHCP Configuration Files

With the traditional Ubuntu DHCP server, important files include:

Main configuration

/etc/dhcp/dhcpd.conf

Edit it:

```
sudo nano /etc/dhcp/dhcpd.conf
```

Example complete configuration:

```
default-lease-time 600;
max-lease-time 7200;

authoritative;

subnet 192.168.1.0 netmask 255.255.255.0 {
    range 192.168.1.100 192.168.1.200;
    option routers 192.168.1.1;
    option subnet-mask 255.255.255.0;
    option domain-name-servers 192.168.1.1 8.8.8.8;
}
```

Interface configuration

The interface used by the DHCP server can be specified in:

```
/etc/default/isc-dhcp-server
```

For example:

```
INTERFACESv4="enp0s3"
```

Find your interface:

```
ip link
```

or:

```
ip addr
```

8. Checking DHCP Configuration

Before restarting the DHCP service, check the configuration:

```
sudo dhcpd -t -cf /etc/dhcp/dhcpd.conf
```

If there are no configuration errors, restart:

```
sudo systemctl restart isc-dhcp-server
```

Check status:

```
sudo systemctl status isc-dhcp-server
```

View recent service messages:

```
sudo journalctl -u isc-dhcp-server
```

9. Automatic Startup of DHCP Server

To make the DHCP server start automatically when Ubuntu boots:

```
sudo systemctl enable isc-dhcp-server
```

Enable and start it together:

```
sudo systemctl enable --now isc-dhcp-server
```

Check:

```
systemctl is-enabled isc-dhcp-server
```

Expected:

enabled

10. Configuration of DHCP Clients

An Ubuntu client can obtain its network configuration automatically through DHCP.

First identify the network interface:

`ip link`

Example:

`enp0s3`

Using NetworkManager, check connections:

`nmcli connection show`

Check the connection's IPv4 method:

`nmcli connection show "Wired connection 1" | grep ipv4.method`

Set DHCP:

`sudo nmcli connection modify "Wired connection 1" ipv4.method auto`

Reactivate the connection:

`sudo nmcli connection up "Wired connection 1"`

Check the assigned address:

`ip addr`

Check the default gateway:

`ip route`

11. Obtaining a DHCP Address from the Command Line

On systems using `dhclient`, you may see commands such as:

`sudo dhclient -v`

To request an address on a particular interface:

```
sudo dhclient -v enp0s3
```

However, modern Ubuntu desktop installations commonly use **NetworkManager**, so nmcli is generally preferable for managing network connections.

12. Manually Configuring DHCP

There are two meanings of "manually configuring DHCP":

A. Manually configuring the DHCP server

Edit:

```
sudo nano /etc/dhcp/dhcpd.conf
```

Example:

```
subnet 192.168.10.0 netmask 255.255.255.0 {  
    range 192.168.10.100 192.168.10.150;  
    option routers 192.168.10.1;  
    option subnet-mask 255.255.255.0;  
    option domain-name-servers 8.8.8.8;  
}
```

Then:

```
sudo dhcpd -t -cf /etc/dhcp/dhcpd.conf  
sudo systemctl restart isc-dhcp-server
```

B. Manually configuring a client to use DHCP

Using NetworkManager:

```
nmcli connection show
```

Then:

```
sudo nmcli connection modify "Wired connection 1" ipv4.method auto  
sudo nmcli connection up "Wired connection 1"
```

Verify:

```
ip addr  
ip route
```

13. DHCP Troubleshooting

If a client does not receive an IP address, check the server:

```
sudo systemctl status isc-dhcp-server
```

Check logs:

```
sudo journalctl -u isc-dhcp-server
```

Check the configuration:

```
sudo dhcpd -t -cf /etc/dhcp/dhcpd.conf
```

Check the network interface:

```
ip link
```

Check IP configuration:

```
ip addr
```

Check routing:

```
ip route
```

Check whether the DHCP server is listening:

```
sudo ss -ltnp | grep ':67'
```

DHCP normally uses:

UDP 67 → DHCP Server

UDP 68 → DHCP Client

Complete Practical Example

Suppose an Ubuntu server has interface enp0s3 and the network is:

Network: 192.168.1.0/24

Server: 192.168.1.10

Gateway: 192.168.1.1

DHCP Range: 192.168.1.100-192.168.1.200

DNS: 8.8.8.8

Install:

```
sudo apt update
```

```
sudo apt install isc-dhcp-server
```

Configure the interface:

```
sudo nano /etc/default/isc-dhcp-server
```

Set:

```
INTERFACESv4="enp0s3"
```

Configure DHCP:

```
sudo nano /etc/dhcp/dhcpd.conf
```

Add:

```
default-lease-time 600;  
max-lease-time 7200;  
authoritative;
```

```
subnet 192.168.1.0 netmask 255.255.255.0 {  
    range 192.168.1.100 192.168.1.200;  
    option routers 192.168.1.1;  
    option subnet-mask 255.255.255.0;  
    option domain-name-servers 8.8.8.8;  
}
```

Test:

```
sudo dhcpd -t -cf /etc/dhcp/dhcpd.conf
```

Start automatically:

```
sudo systemctl enable --now isc-dhcp-server
```

Check:

```
sudo systemctl status isc-dhcp-server
```

On the client:

```
sudo nmcli connection modify "Wired connection 1" ipv4.method auto  
sudo nmcli connection up "Wired connection 1"
```

Verify:

```
ip addr  
ip route
```

The client should receive an address such as:

```
192.168.1.100
```

Quick Reference

Topic	Command/Configuration
Install DHCP server	<code>sudo apt install isc-dhcp-server</code>
DHCP configuration	<code>/etc/dhcp/dhcpd.conf</code>
Interface configuration	<code>/etc/default/isc-dhcp-server</code>
Find interfaces	<code>ip link</code>
Test DHCP configuration	<code>sudo dhcpd -t -cf /etc/dhcp/dhcpd.conf</code>
Start server	<code>sudo systemctl start isc-dhcp-server</code>
Restart server	<code>sudo systemctl restart isc-dhcp-server</code>
Enable at boot	<code>sudo systemctl enable isc-dhcp-server</code>
Enable + start	<code>sudo systemctl enable --now isc-dhcp-server</code>
Check status	<code>sudo systemctl status isc-dhcp-server</code>
DHCP logs	<code>sudo journalctl -u isc-dhcp-server</code>
Check IP	<code>ip addr</code>
Check gateway	<code>ip route</code>
NetworkManager connections	<code>nmcli connection show</code>
Set client to DHCP	<code>nmcli connection modify "Connection" ipv4.method auto</code>
Activate connection	<code>nmcli connection up "Connection"</code>
DHCP ports	UDP 67/68

Note: `isc-dhcp-server` is useful for learning the traditional DHCP administration concepts in your syllabus. For new production deployments, consider **Kea DHCP**, which is the modern DHCP server from the ISC ecosystem.

explain and describe all topic with command and example using Ubuntu 22.024 linux os system “Case Study: - Installation of Linux, Customization of Linux, Installation of SAMBA, APACHE, TOMCAT, Send MAIL, Postfix, Implementation of DNS, LDAP services, Firewall, Proxy “ ?

Assuming you mean **Ubuntu 22.04 LTS** (not 22.024), below is a practical **Linux Case Study** covering installation, customization, SAMBA, Apache, Tomcat, mail/Postfix, DNS, LDAP, firewall, and proxy services with commands and examples.

Case Study: Linux System Administration Using Ubuntu 22.04 LTS

This case study demonstrates how an Ubuntu Linux server can be installed and configured as a **file server, web server, application server, mail server, DNS server, directory server, firewall, and proxy server**.

Lab example network

Server IP: 192.168.1.10
Network: 192.168.1.0/24
Gateway: 192.168.1.1
Domain: example.local

1. Installation of Linux

Ubuntu 22.04 LTS can be installed from an Ubuntu ISO image.

Basic installation steps

1. Download Ubuntu 22.04 LTS ISO.
2. Create a bootable USB.
3. Boot the computer from USB.
4. Select **Install Ubuntu**.
5. Select keyboard and language.
6. Configure network.
7. Create a user account.
8. Select disk/partitioning.
9. Install the system.
10. Reboot.

After installation, update the system:

```
sudo apt update  
sudo apt upgrade -y
```

Check Ubuntu version:

```
lsb_release -a
```

Example:

```
Distributor ID: Ubuntu  
Description: Ubuntu 22.04 LTS  
Release: 22.04
```

Check IP address:

```
ip addr
```

Check hostname:

```
hostname
```

Set hostname:

```
sudo hostnamectl set-hostname linuxserver
```

2. Customization of Linux

Linux can be customized for different users and server requirements.

Create a user

```
sudo adduser student
```

Add the user to the administrative group:

```
sudo usermod -aG sudo student
```

Install useful tools

```
sudo apt install vim nano curl wget git htop net-tools -y
```

Configure timezone

```
timedatectl
```

Set timezone:

```
sudo timedatectl set-timezone Asia/Kolkata
```

Configure hostname

```
sudo hostnamectl set-hostname linuxserver
```

Check system information

```
hostnamectl
```

```
free -h
```

```
df -h
```

```
lsblk
```

These commands help administrators monitor the customized server.

3. Installation of SAMBA

Samba allows Linux to provide Windows-compatible file and printer sharing using SMB/CIFS.

Install Samba

```
sudo apt update
```

```
sudo apt install samba -y
```

Check service:

```
sudo systemctl status smbd
```

Create a shared directory

```
sudo mkdir -p /srv/share
```

Give appropriate ownership/permissions for a simple lab:

```
sudo chmod 2775 /srv/share
```

Create Samba user

The Linux user must already exist:

```
sudo adduser sambauser
```

Set the Samba password:

```
sudo smbpasswd -a sambauser
```

Configure Samba

Backup the configuration:

```
sudo cp /etc/samba/smb.conf /etc/samba/smb.conf.backup
```

Edit:

```
sudo nano /etc/samba/smb.conf
```

Add:

[Shared]

path = /srv/share

browseable = yes

read only = no

valid users = sambauser

Check configuration:

```
sudo testparm
```

Restart Samba:

```
sudo systemctl restart smbd
```

Enable it at boot:

```
sudo systemctl enable smbd
```

From Windows, the share can typically be accessed as:

```
\\192.168.1.10\Shared
```

4. Installation of Apache Web Server

Apache HTTP Server is commonly used to host websites.

Install:

```
sudo apt install apache2 -y
```

Start and enable:

```
sudo systemctl enable --now apache2
```

Check:

```
sudo systemctl status apache2
```

Check Apache version:

```
apache2 -v
```

The default web directory is:

```
/var/www/html
```

Create a test page:

```
sudo nano /var/www/html/index.html
```

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Ubuntu Web Server</title>
</head>
<body>
  <h1>Welcome to Ubuntu Apache Server</h1>
  <p>Linux system administration case study.</p>
</body>
</html>
```

From another computer, open:

```
http://192.168.1.10
```

5. Installation of TOMCAT

Apache Tomcat is a Java application server commonly used for Java web applications.

Install Java:

```
sudo apt install openjdk-11-jdk -y
```

Check Java:

```
java -version
```

Install Tomcat:

```
sudo apt install tomcat9 -y
```

Start it:

```
sudo systemctl enable --now tomcat9
```

Check:

```
sudo systemctl status tomcat9
```

Tomcat normally listens on port:

8080

Test from a browser:

```
http://192.168.1.10:8080
```

For production, avoid exposing the Tomcat management interface unnecessarily.

6. Send Mail / Mail Server

Linux can provide email services using an **MTA (Mail Transfer Agent)**.

Common components include:

- Postfix — SMTP mail transfer
- Dovecot — IMAP/POP3 mail access
- DNS — especially MX records
- TLS — secure communication

For this case study, Postfix is used.

7. Installation of Postfix

Install:

```
sudo apt update
sudo apt install postfix mailutils -y
```

During installation, select an appropriate mail configuration. For a basic lab, **Internet Site** can be selected.

Set the system mail name, for example:

```
example.local
```

Check service:

```
sudo systemctl status postfix
```

Send a local test mail

```
echo "This is a test email." | mail -s "Ubuntu Mail Test" student
```

Check the mail queue:

```
mailq
```

Postfix configuration is mainly located under:

```
/etc/postfix/
```

Main configuration:

```
/etc/postfix/main.cf
```

Check the effective configuration:

```
postconf -n
```

Important: Running a publicly accessible mail server requires careful configuration for DNS, TLS, authentication, relay restrictions, SPF, DKIM, DMARC, spam protection, and reputation. Do not configure an open relay.

8. Implementation of DNS

DNS (Domain Name System) converts domain names into IP addresses.

Example:

```
www.example.local → 192.168.1.10
```

Install BIND9:

```
sudo apt install bind9 bind9-utils -y
```

Check:

```
sudo systemctl status bind9
```

Create a forward zone configuration:

```
sudo nano /etc/bind/named.conf.local
```

Example:

```
zone "example.local" {  
    type master;  
    file "/etc/bind/db.example.local";  
};
```

Create the zone file:

```
sudo cp /etc/bind/db.local /etc/bind/db.example.local
```

Edit:

```
sudo nano /etc/bind/db.example.local
```

Example:

```
$TTL 604800  
@ IN SOA ns1.example.local. admin.example.local. (  
    2  
    604800  
    86400  
    2419200  
    604800 )
```

```
@ IN NS ns1.example.local.  
ns1 IN A 192.168.1.10  
www IN A 192.168.1.10  
mail IN A 192.168.1.10
```

Check configuration:

```
sudo named-checkconf
```

Check the zone:

```
sudo named-checkzone example.local /etc/bind/db.example.local
```

Restart:

```
sudo systemctl restart bind9
```

Test:

```
dig @192.168.1.10 www.example.local
```

9. LDAP Service

LDAP (Lightweight Directory Access Protocol) provides centralized directory services.

It can store:

- Users
- Groups
- Organizational information
- Authentication-related directory data

A common Linux LDAP implementation is **OpenLDAP**.

Install:

```
sudo apt install slapd ldap-utils -y
```

Configure/reconfigure:

```
sudo dpkg-reconfigure slapd
```

Check service:

```
sudo systemctl status slapd
```

Test LDAP:

```
ldapsearch -x -LLL -H ldap://localhost -b dc=example,dc=local
```

A typical LDAP structure may look like:

```
dc=example,dc=local
├── ou=People
│   ├── uid=student
│   └── uid=teacher
└── ou=Groups
    └── cn=students
```

LDAP is particularly useful in larger organizations where centralized identity information is required.

10. Firewall

A firewall controls incoming and outgoing network traffic.

Ubuntu commonly uses **UFW (Uncomplicated Firewall)** as a simpler firewall management interface.

Check status:

```
sudo ufw status
```

Enable firewall:

```
sudo ufw enable
```

Allow SSH:

```
sudo ufw allow 22/tcp
```

Allow HTTP:

```
sudo ufw allow 80/tcp
```

Allow HTTPS:

```
sudo ufw allow 443/tcp
```

Allow Tomcat if it must be directly accessible:

```
sudo ufw allow 8080/tcp
```

Check rules:

```
sudo ufw status numbered
```

Delete a rule:

```
sudo ufw delete allow 8080/tcp
```

For a server, only open ports that are actually required.

11. Proxy Server

A **proxy server** acts as an intermediary between clients and external servers.

Typical uses include:

- Web access control

- Caching
- Logging
- Bandwidth management
- Network policy enforcement

A common Linux proxy is **Squid**.

Install Squid

```
sudo apt update
sudo apt install squid -y
```

Check:

```
sudo systemctl status squid
```

The main configuration file is:

```
/etc/squid/squid.conf
```

Backup it:

```
sudo cp /etc/squid/squid.conf /etc/squid/squid.conf.backup
```

Edit:

```
sudo nano /etc/squid/squid.conf
```

A simple restricted-network example might use:

```
acl localnet src 192.168.1.0/24
http_access allow localnet
http_access deny all
```

Check configuration:

```
sudo squid -k parse
```

Restart:

```
sudo systemctl restart squid
```

Squid commonly listens on:

```
3128
```

Check:

```
sudo ss -lntp | grep 3128
```

Do not create an open proxy. Restrict proxy access to trusted networks/users.

12. Checking All Services

After installing the services, use systemctl to manage them.

```
systemctl status smbd
systemctl status apache2
systemctl status tomcat9
systemctl status postfix
systemctl status bind9
systemctl status slapd
systemctl status squid
```

To see listening network ports:

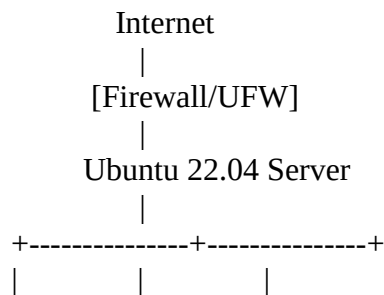
```
sudo ss -ltnp
```

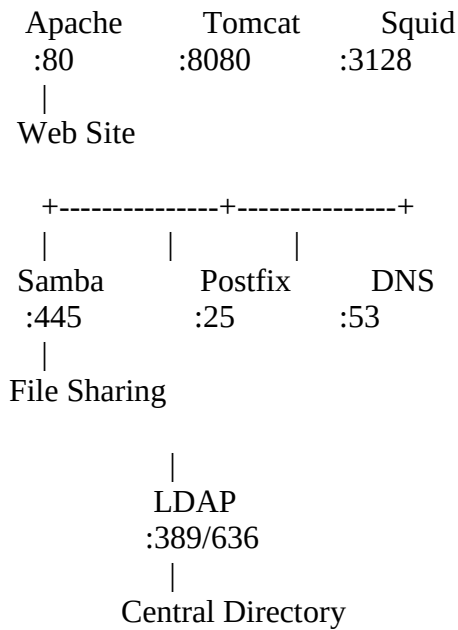
Example services may use:

Service	Typical Port	Purpose
SSH	22	Remote administration
DNS	53	Name resolution
HTTP	80	Web server
HTTPS	443	Secure web
SMTP	25	Mail transfer
LDAP	389	Directory service
LDAPS	636	Secure LDAP
Tomcat	8080	Java web application
Squid	3128	Proxy
SMB	445	File sharing

Complete Case Study Architecture

A simple lab can be organized like this:





Important Security Practices

For a real deployment:

- Use strong passwords.
- Prefer SSH keys over password authentication where appropriate.
- Disable unnecessary services.
- Keep Ubuntu updated:

```
sudo apt update
```

```
sudo apt upgrade
```

- Use UFW/nftables.
- Do not expose LDAP, databases, Tomcat management, Samba, or proxy services unnecessarily to the Internet.
- Use HTTPS/TLS for web applications.
- Secure mail with TLS and proper DNS records.
- Back up configuration files.
- Monitor logs:

```
sudo journalctl -xe
```

- Check listening services:

```
sudo ss -ltnp
```

Final Summary

This case study demonstrates how **Ubuntu 22.04 LTS** can be configured as a complete Linux server platform:

Linux Installation → Customization → Samba File Server → Apache Web Server → Tomcat Application Server → Postfix Mail Server → BIND9 DNS → OpenLDAP Directory Server → UFW Firewall → Squid Proxy Server.

These services represent the major components commonly studied in a Linux system-administration laboratory and provide practical experience with installation, configuration, service management, networking, and security